

System Architecture Overview

1. Architecture Summary

ExpenseTrack is a web-based personal finance tool that helps users manage their spending by logging expenses and viewing detailed reports. The system uses a modular monolith architecture, meaning all components—frontend, backend, and database—are designed to work together as a single, cohesive application while maintaining clear internal separation of concerns.

This architecture was chosen because it balances simplicity and scalability for a mid-sized application like ExpenseTrack. It allows fast development, easy testing, and straightforward deployment using Docker, making it ideal for both local development and future cloud deployment. The backend handles business logic and data processing, the frontend delivers a responsive user interface, and the database securely stores user and expense information—all working together seamlessly.

2. Key System Components

Backend API

- **Role:** Central hub for all application logic and data handling.
- **Responsibility:** Manages user authentication, processes expense entries, enforces security, and generates spending reports.
- **Interaction:** Receives requests from the frontend, interacts with the database to store or retrieve data, and returns structured responses in JSON format. Secured using JWT tokens to ensure only authenticated users can access sensitive features.

Frontend UI

- **Role:** User-facing interface for interacting with the app.
- **Responsibility:** Displays forms for logging in, adding expenses, viewing lists, and showing visual reports using charts.
- **Interaction:** Communicates with the Backend API over HTTPS to send user actions (like adding an expense) and receive updated data (like a list of expenses). Built to be responsive, so it works well on both desktop and mobile devices.

Database (PostgreSQL)

- Role: Secure, reliable storage for all user and expense data.
- Responsibility: Stores user accounts, encrypted passwords, and individual expense records with categories, amounts, and dates.
- Interaction: Accessed exclusively by the Backend API using safe, parameterized queries to prevent security risks like SQL injection.

External Libraries (Chart.js)

- Role: Visual representation of spending data.
 - Responsibility: Converts numerical report data into easy-to-understand charts such as pie and bar graphs.
 - Interaction: Runs in the browser as part of the Frontend UI, receiving data from the backend's report endpoints.
-

3. Data Flow

1. User Registration/Login:

The user submits their email and password via the Frontend UI. This data is sent securely to the Backend API, which verifies credentials, creates an account (if signing up), or generates a secure JWT token (if logging in).

2. Adding an Expense:

After logging in, the user fills out an expense form. The Frontend sends this data to the Backend API, which validates it, associates it with the user, and saves it in the database.

3. Viewing Expenses:

The Frontend requests the user's expense list from the Backend API. The backend retrieves the data from the database and returns it. The frontend then displays the list in a clean, organized format.

4. Generating Reports:

When the user requests a report, the frontend calls the report endpoint. The backend aggregates expense data by month or category, calculates totals, and returns summarized data. The frontend uses Chart.js to display this as visual charts.

5. Logging Out:

The frontend invalidates the current session, ensuring the JWT token is no longer

used, thus protecting user privacy.

4. Database Design

The system uses PostgreSQL, a robust and secure relational database, to store structured data. It ensures data integrity and supports efficient querying for reports.

Major Data Entities:

- User: Stores user information including a unique ID, email, and securely hashed password.
- Expense: Records each expense with an amount, category (e.g., Food, Transport), date, optional notes, and a link to the owning user.

Purpose:

The database supports core features like secure login, personal expense tracking, and report generation. Relationships between users and their expenses ensure data privacy—users only see their own records.

5. External Integrations

- Chart.js
- Purpose: Render interactive and visually appealing spending reports.
- Integration: Embedded directly into the Frontend UI via a standard JavaScript package. It does not require external API calls and works entirely in the user's browser using data from the backend.

No third-party payment or cloud services are used, keeping the system self-contained and secure.

6. Scalability & Performance

While designed as a personal expense tracker, the system is built to handle growth:

- Scalability: The use of Docker and modular design allows the application to scale horizontally in the future. For example, multiple backend instances can be added

behind a load balancer if user demand increases.

- **Caching:** Redis is included in the setup to support future caching of frequent queries (e.g., monthly reports), reducing database load and improving response speed.
 - **Performance:** The frontend is optimized to load quickly and handle moderate amounts of expense data efficiently. For large datasets, pagination or filtering can be added later to maintain smooth performance.
-

7. Security Considerations

Security is a top priority to protect sensitive financial data:

- **Authentication:** Users log in using secure JWT (JSON Web Tokens), which are short-lived and automatically expire after 30 minutes for safety.
 - **Data Protection:** Passwords are never stored in plain text—they are encrypted using bcrypt before being saved in the database.
 - **API Security:** All sensitive endpoints require a valid JWT token. Input validation is enforced on both frontend and backend to prevent malicious data entry.
 - **Secure Communication:** All data between the frontend and backend is transmitted over HTTPS in production, preventing eavesdropping.
-

8. Deployment Overview

The entire system is containerized using Docker and orchestrated with Docker Compose, enabling simple and consistent deployment on any machine.

Components in Deployment:

- **Backend Service:** Runs the FastAPI application on port 8000.
- **Frontend Service:** Serves the React app on port 3000.
- **Database (PostgreSQL):** Runs in its own container, storing all persistent data.
- **Redis:** Optional cache service, ready for future performance enhancements.

With a single command (`docker-compose up`), all services start together, connect automatically, and are ready to use—making setup fast and error-free for developers and end users alike.

9. Future Improvements

The current version provides a solid foundation. Future enhancements could include:

- Email Notifications: Alert users about monthly spending limits.
- Budgeting Tools: Allow users to set monthly budgets per category.
- Data Export: Enable download of expenses as CSV or PDF.
- Mobile App: Build a native mobile version using React Native.
- Multi-Currency Support: Track expenses in different currencies.
- Cloud Sync: Store data in the cloud for access across devices.

These features can be added incrementally without disrupting the existing system, thanks to the clean architecture.